BÖLÜM 8 / MODULE 8

Dilimler (Slices), Testler ve İterasyon Döngüsü

Slices, Tests, and the Iteration Loop

Use-Case 3.0'ın 10. İlkesi: Sistemi dilimler halinde inşa etmek. Temel akış ilk dilimdir; alternatif akışlar iş önceliğine göre dilimlenir. İlke 14: 'Gerçeği Testler Taşır'. AIUP test piramidi, TC-XXX uçtan uca yolculukları, birebir (literal) test verisi ve BPMN iş süreçlerinden test türetme döngüsü.

Use-Case 3.0's 10th Principle: Build the system in slices. The basic flow is Slice 1; alternative flows are sliced in business-priority order. Principle 14: 'Tests Carry the Truth'. The AIUP test pyramid, TC-XXX end-to-end journeys, literal test data discipline, and deriving test cases from BPMN processes.

43

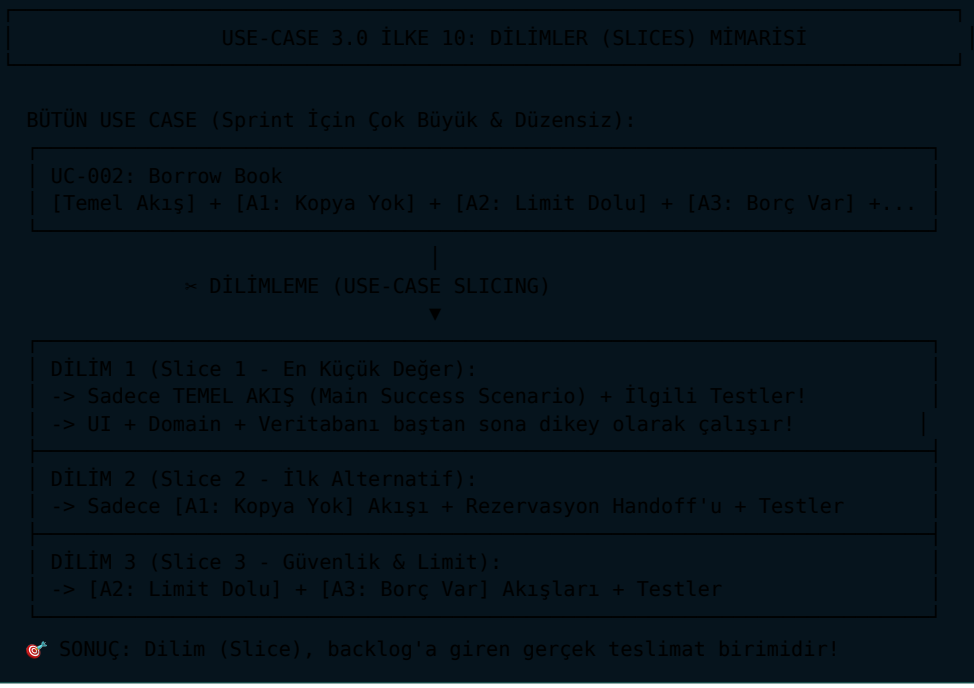
BÖLÜM: DILIMLER (SLICES), TESTLER VE İTERASYON DÖNGÜSÜ | MODULE: SLICES, TESTS, AND THE ITERATION LOOP

10. İlke: Sistemi Dilimlerle İnşa Etmek (Use-Case Slices)

Principle 10: Slices – Slicing the System into Deliverable Units

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Kocaman bir doğum günü pastasını bir lokmada ağzınıza atamazsınız, boğulursunuz! Pastayı dilim dilim kesersiniz. İlk dilim sadece kekin ve kremanın tadını verir (Temel akış). Sonraki dilimlerde çilekler ve çikolatalar gelir (Alternatif akışlar). İşte use case pastanın tamamıdır, sprint'e giren şey ise o tek bir lezzetli dilimdir (Slice)!

🇬🇧 ENGLISH (ELI5)

You cannot swallow an entire multi-layered birthday cake in one bite; you'd choke! You slice it cleanly. Slice 1 gives you the foundational sponge cake and frosting (Basic Flow). Subsequent slices bring strawberries and chocolate chips (Alternative Flows). The use case is the whole cake; the actionable backlog ticket for your sprint is the vertical slice!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Use-Case 3.0'ın 10. İlkesi (Build the System in Slices): Ivar Jacobson ve Ian Spence'in klasik use case metodolojisine kattığı en devrimci ilke budur. Bir use case, bir hedefe ulaşmanın TÜM yollarını (başarı, istisnalar, hatalar) içerdiği için tek bir sprint veya tek bir PR için fazla büyüktür ve dengesiz boyutlardadır. Teslimat birimi use case değil, 'Use-Case Slice'dır. Bir dilim; use case içindeki bir veya birkaç yolu, o yolları icra eden tasarımı, kodu ve testleri içeren dikey, bağımsız ve test edilebilir bir pakettir. Bu sayede use case'ler Scrum sprint backlog'larına, Kanban sütunlarına ve SAFe trenlerine mükemmel uyum sağlar. AI Unified Process'te bir şartnamenin Status alanı bu dilimleme yaşam döngüsünü yönetir: Temel akış kodlanıp testten geçtiğinde use case 'Implemented/Tested' aşamasına geçer; kalan alternatif akışlar şartnamede beklerken sistem güvenle çalışır durumda kalır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Principle 10: Building the System in Slices: Jacobson and Spence added the 10th principle to bridge classical use cases with agile delivery. Because a use case captures every permutation of achieving a goal, it is irregularly sized and too monolithic to flow smoothly through a sprint board. The fundamental unit of delivery is the use-case slice: one or more paths through a use case, accompanied by the design, production code, and automated tests that implement and verify them. Slices start at the trigger, conclude at the postconditions, and provide immediate, testable business value. Slices map directly to user stories or sprint backlog items. In the AI Unified Process, this additive structure allows teams to generate Slice 1 via `/implement UC-002`, review the running UI with stakeholders, and then progressively generate slices for alternative flows without architectural churn.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Birinci Dilim (Slice 1: Sadece Temel Akış) Uygulaması
package com.maang.aiup.slices;

import java.time.LocalDate;

public class UseCaseSliceOneDemo {
    // DİLİM 1: Sadece temel akış verileri ve başarısı (Alternatifler henüz yok!)
    public record BorrowSliceOneResult(String bookId, String memberId, LocalDate dueDate) {}

    public static class BorrowBookSliceOneService {
        public BorrowSliceOneResult executeBasicFlow(String bookId, String memberId) {
            // Dilim 1 kapsamında sadece mutlu yol çalışır:
            LocalDate dueDate = LocalDate.now().plusDays(28);
            System.out.println("📦 Slice 1 Başarılı: Kitap (" + bookId + ") üyeye (" + memberId + ") verildi.");
            return new BorrowSliceOneResult(bookId, memberId, dueDate);
        }
    }

    void main() {
        var sliceOne = new BorrowBookSliceOneService();
        var result = sliceOne.executeBasicFlow("BK-42", "MEM-007");
        System.out.println("✅ Dilim 1 Dikey Olarak Çalıştı: " + result);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir use case'in tüm akışlarını (happy path + 8 alternatif akış) tek bir seferde AI'a kodlatmaya çalışmayın! Temel akışı ilk dilim olarak üretip paydaşa gösterin, geri bildirim alın, sonra diğer akışları ekleyin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Resist the urge to generate an entire use case (happy path + 8 branches) in a single massive prompt. Implement Slice 1 first, get visual feedback, then add alternative branches incrementally.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Teslimatın temel birimi use case değil, dikey ve test edilebilir 'use-case slice'dır.
- Dilim 1 daima temel akıştır (happy path); en hızlı iş değerini ve geri bildirimi o sağlar.
- Dilimleme sayesinde use case'ler Scrum, Kanban ve CI/CD akışlarına pürüzsüz entegre olur.

🎯 Key Architectural Takeaways

- The unit of delivery is the use-case slice, not the entire use case document.
- Slice 1 is always the basic flow, delivering immediate feedback and functional baseline.
- Slicing enables seamless integration of formal use cases into Scrum, Kanban, and modern CI/CD.

44

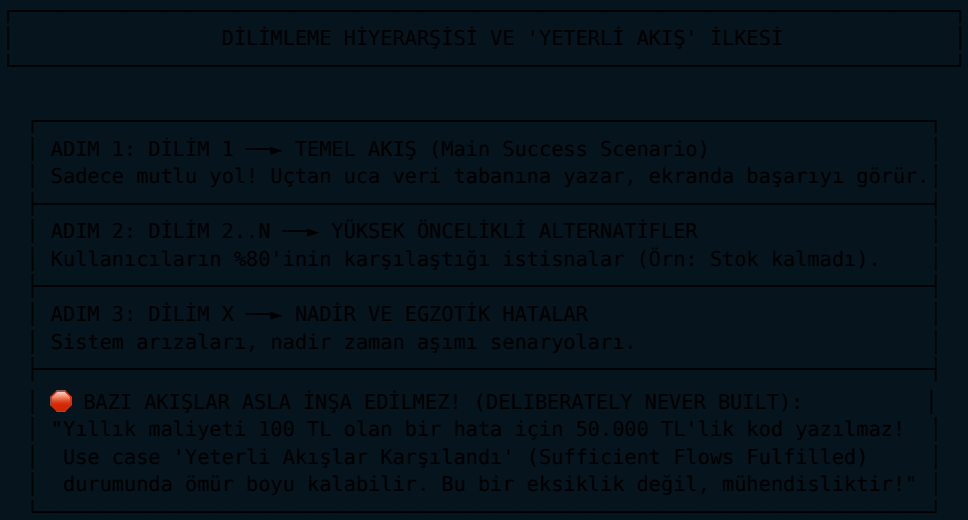
BÖLÜM: DILIMLER (SLICES), TESTLER VE İTERASYON DÖNGÜSÜ | MODULE: SLICES, TESTS, AND THE ITERATION LOOP

Dilimleme Hiyerarşisi: Önce Temel Akış, Sonra Alternatifler (Bazıları Asla Yapılmaz!)

The Slicing Order: Basic Flow First, Alternatives by Priority, Some Never Built

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Yeni açılan bir kahve dükkânını düşünün: İlk gün sadece sıcak kahve satarlar (Temel akış). İkinci hafta vanilyalı ve karamelli şuruplar gelir (Öncelikli alternatifler). 'Organik Madagaskar vanilyalı yulaflı latte'yi ise yılda bir kişi sorar diye menüye hiç koymazlar (Asla inşa edilmeyen akış!). Her şeyi aynı anda yapmaya çalışmak dükkânı batırır!

🇬🇧 ENGLISH (ELI5)

Consider opening a new coffee shop: On Day 1, they only brew classic hot coffee (Basic Flow). By Week 2, they introduce caramel and hazelnut syrups (High-priority alternatives). But 'Organic Madagascar Vanilla Bean Oat Milk Cappuccino' is requested once a year, so they never add it to the menu (Deliberately never built!). Slicing prevents capital exhaustion.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Dilimleme Sırası ve 'Yeterli Akış' Felsefesi: Use-Case 3.0 ve Martinelli'nin metodolojisinde dilimleme sırası katı bir mantığa bağlıdır: 1) Dilim 1 Temel Akış: Sistemin en önemli use case'inin temel akışı, tüm mimari omurgayı (UI, servis, DB, güvenlik) doğrulamak için yeterlidir. 2) Sonraki Dilimler: En sık rastlanan ve iş değeri en yüksek alternatif akışlar sırayla dilimlenir. 3) 'Bazı Akışlar Asla İnşa Edilmez': Geleneksel waterfall zihniyeti şartnamedeki her satırın kodlanmasını şart koşar. Oysa Use-Case 3.0'ın en gerçekçi durumu 'Yeterli Akışlar Karşılandı' (Sufficient Flows Fulfilled) durumudur. Sistem ticari olarak hedefine ulaşmışsa, 5 yılda bir olabilecek çok nadir bir istisna için kod yazmak ve bakım maliyeti üstlenmek israftır. O akış şartnamede analiz edilmiş olarak bekler; manuel operasyonel süreçle idare edilir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Slicing Order and the 'Sufficient Flows Fulfilled' State: Slicing follows an immutable cadence derived from the additive spec architecture: 1) Slice 1 is strictly the Basic Flow. Implementing the happy path end-to-end verifies the architectural backbone (REST endpoints, persistence, transaction boundaries, UI state) and yields early stakeholder validation. 2) Successive Slices incorporate alternative flows sorted strictly by business priority and risk mitigation. 3) Deliberately Unbuilt Flows: A vital tenet of Use-Case 3.0 is that not every analyzed flow must be implemented. A use case can intentionally reside in 'Sufficient Flows Fulfilled' indefinitely. Engineering rare edge-case automation whose cost exceeds manual operational fallback is anti-pattern waste. The specification documents the path; the business consciously opts not to cut the slice.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Dilimlerin Öncelik Sırasıyla Eklemeli (Additive) Olarak Büyütülmesi
package com.maang.aiup.slices;

public class SlicingEvolutionDemo {
    // Aşama 1: Sadece Temel Akış (Slice 1)
    public record BorrowRequest(String bookId, int availableCopies, int currentLoans) {}

    public static class BorrowFlowHandler {
        // Slice 1: Temel Başarı
        public String executeSlice1(BorrowRequest req) {
            return "✅ Kitap ödünç verildi.";
        }

        // Slice 2 Eklenir: A1 (Kopya Yok) ve A2 (Limit Doldu)
        public String executeSlice2(BorrowRequest req) {
            if (req.availableCopies() <= 0) {
                return "⚠ Alternatif A1: Kopya yok, rezervasyona yönlendirildi.";
            }
            if (req.currentLoans() >= 5) {
                return "❌ Alternatif A2: Limit dolu (5 kitap), işlem engellendi.";
            }
            return executeSlice1(req);
        }
    }

    void main() {
        var handler = new BorrowFlowHandler();
        var normalReq = new BorrowRequest("BK-1", 3, 2);
        var noStockReq = new BorrowRequest("BK-2", 0, 1);

        System.out.println("1. İstek: " + handler.executeSlice2(normalReq));
        System.out.println("2. İstek: " + handler.executeSlice2(noStockReq));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Temel akışı canlıya almadan veya paydaşa onaylatmadan alternatif akışları kodlamaya başlamayın; temel akışta yapılacak bir UI değişikliği tüm alternatifleri çöp edebilir.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never begin coding alternative flows before the basic flow is reviewed by stakeholders; a fundamental change in happy path UI invalidates all downstream branches.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Dilimleme sırası sabittir: Önce temel akış, sonra en sık rastlanan alternatifler.
- Her alternatif akışı kodlamak zorunda değilsiniz; bazıları operasyonel olarak çözülür.
- 'Yeterli Akışlar Karşılandı' durumu bir use case için tamamen geçerli bir bitiş durumudur.

🎯 Key Architectural Takeaways

- The slicing order is fixed: basic flow first, followed by frequent alternatives.
- Not all alternative flows must be coded; manual workarounds are often economically sound.
- 'Sufficient Flows Fulfilled' is a valid, mature end-state for a production use case.

45

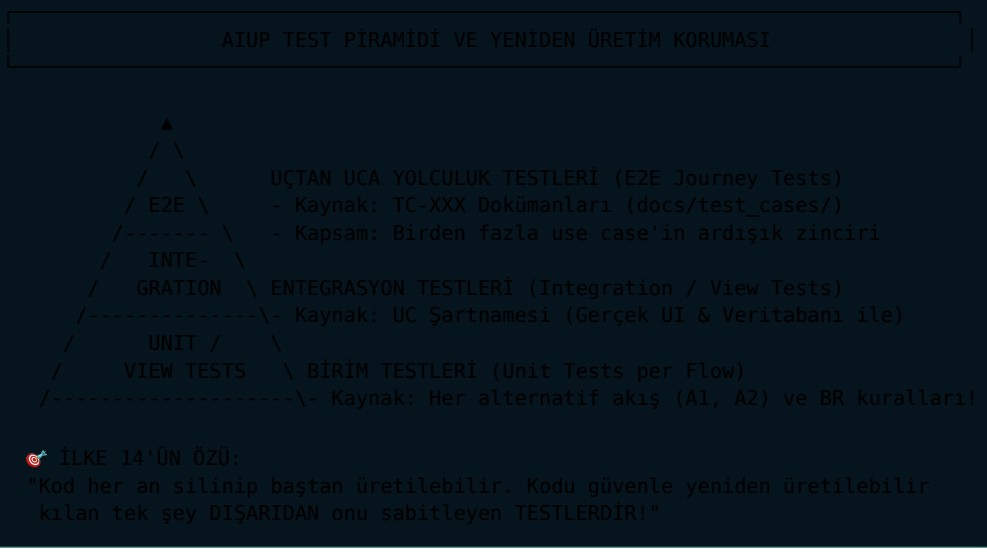
BÖLÜM: DILIMLER (SLICES), TESTLER VE İTERASYON DÖNGÜSÜ | MODULE: SLICES, TESTS, AND THE ITERATION LOOP

14. İlke: Gerçeği Testler Taşır (AIUP Test Piramidi)

Principle 14: Tests Carry the Truth – The AIUP Test Pyramid

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Lunaparktaki hız trenini (rollercoaster) düşünün: Trenin vagonunu istediğiniz renge boyayabilir, koltuklarını değiştirebilirsiniz. Ama vagonun raydan çıkmasını engelleyen şey altındaki çelik kilitlerdir! Testler o çelik kilitlerdir. AI kodu baştan üretse bile testler trenin raydan çıkıp uçuruma yuvarlanmasını engeller!

🇬🇧 ENGLISH (ELI5)

Think of a roller coaster: You can repaint the coaster cars or upgrade the upholstery anytime. But what keeps the car from flying off the tracks at 100 mph are the steel under-friction safety wheels! Automated tests are those steel clamps. Even if an AI completely regenerates the engine, the tests guarantee the car stays on track.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

14. İlke: Gerçeği Testler Taşır (Tests Carry the Truth): Use-Case 3.0'daki en çarpıcı cümle şudur: 'Test senaryoları, bir use case'in anlatısından bile daha önemli olan en kritik parçasıdır; çünkü use case'in ne zaman başarıyla tamamlandığını sadece testler tanımlar!' AI Unified Process bu tavsiyeyi yapısal bir zorunluluğa dönüştürür. 4. İlke 'Test-Protected' (Testle Korunan): Kod her an baştan üretilebilen geçici bir derleme çıktısıdır. Eğer bir sistemin davranışı kodun dışında bir mekanizma tarafından sabitlenmemişse, o sistemi AI ile yeniden üretmek kumar oynamaktır! Test piramidi doğrudan şartname zincirine bağlanır: 1) Birim/Akış Testleri: Şartnamedeki her bir doğrulama adımını ve her bir alternatif akışı (A1, A2) tek tek sınar. 2) Entegrasyon Testleri: Bir use case'in tamamını gerçek arayüz ve servislerle test eder. 3) Yolculuk Testleri (TC-XXX): Birden fazla use case'i bağlayan iş süreçlerini test eder.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Principle 14: Tests Carry the Truth & The AIUP Test Pyramid: Use-Case 3.0 asserts boldly: 'The test cases are the most important part of a use case's description, more important even than the narrative.' In the AI Unified Process, this transitions from agile guidance to absolute architectural necessity. Principle 4 (Test-Protected) establishes that because code can be generated, refactored, or discarded at will, intent must be pinned by artifacts outside the code. The tests freeze observable behavior. The test hierarchy aligns cleanly with the artifact chain: 1) Unit/Flow Tests: Derived directly from the use case validations and alternative flows (BR-XXX). 2) Integration Tests: Drive an entire use case through the presentation layer. 3) Journey Tests (TC-XXX): Verify multi-use-case lifecycles where state transitions cross use case boundaries.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: Alternatif Akışları Doğrulayan Test-Protected Mimari
package com.maang.aiup.testing;

import java.util.Optional;

public class TestCarriesTruthDemo {
    public record BorrowResult(boolean success, String message) {}

    public static class BorrowValidator {
        public static BorrowResult validate(int availableCopies, int currentLoans, boolean hasOverdue)
{
            // BR-004: Kopya kontrolü -> Alternatif A1
            if (availableCopies <= 0) return new BorrowResult(false, "A1: Kopya kalmadı.");
            // BR-005: Limit kontrolü -> Alternatif A2
            if (currentLoans >= 5) return new BorrowResult(false, "A2: Limit aşıldı.");
            // BR-006: Gecikmiş borç -> Alternatif A3
            if (hasOverdue) return new BorrowResult(false, "A3: Gecikmiş borç var.");

            return new BorrowResult(true, "Ödünç başarılı.");
        }
    }

    // TEST MOTORU (Şartnamenin bekçisi)
    void main() {
        // Test 1: A1 Akışını sına
        var resA1 = BorrowValidator.validate(0, 1, false);
        assert !resA1.success() && resA1.message().contains("A1") : "A1 Testi Başarısız!";

        // Test 2: A2 Akışını sına
        var resA2 = BorrowValidator.validate(2, 5, false);
        assert !resA2.success() && resA2.message().contains("A2") : "A2 Testi Başarısız!";

        // Test 3: Başarı Yolunu sına
        var resOk = BorrowValidator.validate(2, 2, false);
        assert resOk.success() : "Mutlu Yol Testi Başarısız!";

        System.out.println("✅ Tüm akış testleri geçti! Şartname koruma altındadır.");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Testi yazılmamış bir use case şartnamesi yalnızca bir temenniden ibarettir! Her alternatif akışın en az bir otomatik testi olmak zorundadır.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

A use case without automated test cases is merely a wish. Every alternative flow must have at least one automated assertion guarding it against regression.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Kod geçicidir, test kalıcıdır: Sistemi güvenle yeniden üretilebilir kılan testlerdir.

• Şartnamedeki her bir doğrulama (validates) ve alternatif akış (A1, A2) için test yazılır.

• Yolculuk testleri (TC-XXX) use case'ler arasındaki durum ve veri akışını denetler.

🎯 Key Architectural Takeaways

• Code is ephemeral; tests are enduring. Automated tests make regeneration safe.

• Every validation step and alternative flow in the spec maps to a specific test case.

• Journey tests (TC-XXX) verify cross-use-case boundaries and continuous state progression.

46

BÖLÜM: DILIMLER (SLICES), TESTLER VE İTERASYON DÖNGÜSÜ | MODULE: SLICES, TESTS, AND THE ITERATION LOOP

Test Senaryoları (TC-XXX): Use Case'leri Birbirine Bağlayan Uçtan Uca Yolculuklar

Test Cases (TC-XXX): End-to-End Journeys Chaining Multiple Use Cases

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────┐
│              TC-XXX: UÇTAN UCA İŞ YOLCULUĞU TABLOSU      │
└──────────────────────────────────────────────────────────┘

TC-001: Yeni Üye Olma ve İlk Kitabı Ödünç Alma Yolculuğu

┌─────┬──────────────────────────┬──────────────────────┬──────────────────┐
│  #  │ Adım Açıklaması          │ Kullanılan Test Verisi│ İlgili Use Case  │
├─────┼──────────────────────────┼──────────────────────┼──────────────────┤
│  1  │ Kullanıcı üye kaydı açar │ "bozkurt@maang.com"  │ UC-001 Register  │
│  2  │ E-posta onayını tamamlar │ Onay Linki / Token   │ UC-001 Register  │
│  3  │ Katalogda kitap arar     │ "Clean Architecture" │ UC-003 Search    │
│  4  │ Kitabı ödünç alır        │ Kitap ID: "BK-101"   │ UC-002 Borrow    │
│  5  │ Ödünç listesini denetler │ Vade: 28 gün sonra   │ UC-004 My Loans  │
└─────┴──────────────────────────┴──────────────────────┴──────────────────┘

🎯 MİMARİ İLKE:
- Tek tek use case'ler detayları (kolonlar, hata mesajları) test eder.
- TC-XXX ise DİKİŞ YERLERİNİ (Seams) test eder: Durumun bir use case'den
  diğer use case'e pürüzsüz aktarıldığını doğrular!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir bayrak yarışını (atletizm) düşünün: 4 farklı koşucu vardır. Her koşucu kendi 100 metresini harika koşabilir (Bireysel Use Case). Ama en kritik an, sopayı bir sonraki koşucunun eline düşürmeden vermektir (Dikiş Noktası / Seams)! TC-XXX testleri, sopanın elden ele düşmeden geçtiğini kontrol eden başhakemdir!

🇬🇧 ENGLISH (ELI5)

Think of a 4x100m track relay race: Four runners sprint their own legs (Individual Use Cases). Each runner may run exceptionally well alone. But the most catastrophic failure happens at the baton handoff (The Seam)! A TC-XXX test case acts as the head referee, ensuring the baton passes flawlessly from runner to runner across the entire lap.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

TC-XXX Uçtan Uca Yolculuk Dokümanlarının Mimarisi: Bir use case tek bir aktörün tek bir oturumda ulaştığı deniz seviyesi hedefidir (Sea-level). Ancak gerçek hayatta müşteriler ardışık birçok hedefi zincirleme icra eder: 'Kayıt Ol -> Doğrula -> Ara -> Sepete Ekle -> Öde -> Kargo Takip Et'. Cockburn'ün 'Uçurtma Seviyesi' (Kite-level) dediği bu süreçler AIUP içinde `docs/test_cases/TC-XXX-\<name>.md` dosyaları olarak formalize edilir. TC dokümanı tek bir akış tablosudur. Her satır bir adımı, o adımda kullanılan somut test verisini ve zincirlenen use case dosyasını (`UC-001`, `UC-002`) barındırır. Bu ayrım katı bir iş bölümü sağlar: 1) Use case seviyesindeki testler iç detaylara (kolonlar, validasyonlar, hata mesajları) odaklanır. 2) TC yolculuk testleri ise dikiş yerlerini (seams) denetler; yani UC-001'in oluşturduğu üye ID'sinin UC-002 tarafından hatasız şekilde tanınıp kullanılabildiğini garanti eder.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Architecture of TC-XXX End-to-End Journey Specifications: While a use case encapsulates a single sea-level user goal achieved in one sitting, business workflows span cohesive multi-step journeys (e.g., onboarding, checkout, fulfillment). Cockburn designated these 'kite-level' summary goals. In the AI Unified Process, kite-level workflows are codified as test journey artifacts in `docs/test_cases/TC-XXX-\<name>.md`. A TC document consists of a structured tabular progression where each row declares: the step description, literal input/output test data, and a hyperlink to the executed use case specification. This establishes a clean demarcation of testing concerns: individual use case specs own flow-level minutiae (validation messages, column sort orders), while journey tests own system seams—ensuring data and state flow seamlessly across use case boundaries.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Use Case'ler Arası Durum Aktarımını Doğrulayan Yolculuk Testi
package com.maang.aiup.journeys;

public class CrossUseCaseJourneyTest {
    // Adım 1: Kayıt Use Case Çıktısı
    public record RegisteredMember(String id, String email, boolean active) {}

    // Adım 2: Ödünç Use Case Çıktısı
    public record BorrowedReceipt(String loanId, String memberId, String bookId) {}

    public static class RegistrationService {
        public RegisteredMember register(String email) {
            return new RegisteredMember("MEM-99", email, true);
        }
    }

    public static class BorrowService {
        public BorrowedReceipt borrow(RegisteredMember member, String bookId) {
            if (!member.active()) throw new IllegalStateException("Üye aktif değil!");
            return new BorrowedReceipt("LN-1001", member.id(), bookId);
        }
    }

    void main() {
        // TC-001 Yolculuk İcrası:
        var regService = new RegistrationService();
        var borrowService = new BorrowService();

        // 1. Dikiş Noktası: Kayıt ol
        var member = regService.register("bozkurt@maang.com");
        System.out.println("TC-001 Adım 1 Başarılı: " + member);

        // 2. Dikiş Noktası: Kayıtlı üyeyi ödünç servisine geçir
        var receipt = borrowService.borrow(member, "BK-Clean-Arch");
        System.out.println("TC-001 Adım 2 Başarılı: " + receipt);

        assert receipt.memberId().equals("MEM-99") : "Dikiş hatası! Üye ID aktarılamadı.";
        System.out.println("✅ TC-001 Uçtan Uca Yolculuk Başarıyla Doğrulandı!");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Yolculuk testlerinde (TC-XXX) tablo kolonlarını veya arayüz renklerini yeniden test etmeye kalkmayın! Yolculuk testinin tek görevi use case'ler arasındaki durum aktarımını sınamaktır.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never re-assert table columns or UI styling inside end-to-end journey tests. Journey tests own the seams between use cases, not internal UI presentation details.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• TC-XXX dokümanları, birden fazla use case'i birleştiren uçtan uca yolculukları tanımlar.

• Yolculuk testleri 'dikiş yerlerini' (seams) denetler; durumun use case'ler arası geçişini doğrular.

• Her satırda somut test verisi ve tetiklenen use case referansı yer alır.

🎯 Key Architectural Takeaways

• TC-XXX documents specify end-to-end user journeys that chain multiple use cases.

• Journey tests verify system seams, ensuring state propagates across use cases flawlessly.

• Each journey step declares literal test data and links to the invoked use case specification.

47

BÖLÜM: DILIMLER (SLICES), TESTLER VE İTERASYON DÖNGÜSÜ | MODULE: SLICES, TESTS, AND THE ITERATION LOOP

Test Verisi Disiplini: Somut Veri ('Acme Corp', '5') vs Ölümcül Yer Tutucular

Test Data Discipline: Literal Test Data vs Deadly Generic Placeholders

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────┐
│              SOMUT TEST VERİSİ VS ÖLÜMCÜL YER TUTUCULAR  │
└──────────────────────────────────────────────────────────┘

❌ ÖLÜMCÜL YER TUTUCULAR (GENERIC PLACEHOLDERS):
- "Kullanıcı geçerli bir ad ve e-posta girer."
- "Sistem siparişi uygun miktarda oluşturur."
💥 AI Ne Yapar?: 'Geçerli e-posta' ne demek? 'test@test' mi, 'a@b.c' mi?
                 AI kendi kafasından veri uydurur ve test kırılganlaşır!

✅ BİREBİR SOMUT TEST VERİSİ (LITERAL TEST DATA):
- "Kullanıcı Şirket Adı olarak 'Acme Corp', Adet olarak '5' girer."
- "Kullanıcı e-posta olarak 'sarah.connor@cyberdyne.com' yazar."

🎯 MİMARİ KURAL:
- İnsan test mühendisi yer tutucuların etrafından doğaçlama yapabilir.
- Kod üreten yapay zekâ DOĞAÇLAMA YAPAMAZ!
- Bıraktığınız her yer tutucu, AI'ın ürettiği bir halüsinasyona dönüşür!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir terziye takım elbise diktireceğinizi düşünün: Terziye 'Bana uygun boyda bir pantolon dik' derseniz (Yer tutucu!), terzi boyunuzu tahmin eder ve pantolon ya dizinizde kalır ya yerde sürünür! Ama 'Bel 82 cm, paça boyu 104 cm' derseniz (Somut veri!), pantolon üstünüze tam oturur. Testte de 'geçerli veri' değil, tam santimetreleri vermelisiniz!

🇬🇧 ENGLISH (ELI5)

Imagine ordering a custom tailored suit: If you tell the tailor 'Sew me a properly sized jacket' (Generic placeholder!), they guess your dimensions and the sleeves hang down past your knees! But if you declare 'Chest 40 inches, sleeve 33 inches' (Literal data!), it fits like a glove. Automated tests cannot guess; they demand exact dimensions.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Test Verisinde Birebirlik (Literal Test Data) İlkesi: Şartname ve test dokümanlarında mühendislerin düştüğü en büyük tuzaklardan biri soyut yer tutucudır ('geçerli bir müşteri', 'örnek bir ürün', 'uygun bir tarih'). İnsan test uzmanı bu metni okuduğunda zihninde 'John Doe', 'Laptop', '2026-05-01' gibi veriler uydurabilir. Ancak yapay zekâ destekli üretim motorları (`/test-case`) soyut kavramları rastgele doldurur. Bir çalıştırmada boşluk 'user1' ile doldurulurken diğerinde 'test@invalid' ile doldurulabilir ve testler tutarsızlaşır (flaky tests). Martinelli'nin altın kuralı: 'Test verisi harfi harfine somut (literal) olmalıdır.' Test tablosunda müşteri adı 'Acme Corp', ürün adı 'Widget-X', miktar '5', tarih '2026-10-20' olarak açıkça yazılmalıdır. Bu sayede üretilen test kodları deterministik, tekrarlanabilir ve sıfır varsayımla çalışır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Literal Test Data Mandate: In traditional testing, human QA engineers tolerate placeholder abstractions like 'a valid customer' or 'sufficient balance' because human intuition bridges the gap with impromptu mock values. Generative AI agents possess no such intuition. When presented with placeholder phrasing, an AI implementer invents arbitrary mock data—varying between runs, introducing edge-case boundary mutations, and creating flaky test suites. Martinelli insists upon literalism: 'Acme Corp, Widget, 5, never a valid customer.' Specifying explicit string literals, deterministic integers, and concrete RFC-compliant dates ensures that generated unit and integration tests compile with identical, deterministic assertions across every single regeneration cycle.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Deterministik Somut Test Verisi ile Güvenli Test İcrası
package com.maang.aiup.testdata;

public class LiteralTestDataDemo {
    // ❌ YANLIŞ: Soyut yer tutucu (AI kafasına göre mock nesnesi üretir)
    // var customer = createAnyValidCustomer();

    // ✅ DOĞRU: Birebir Somut Veri (Literal Test Data)
    public record CustomerOrder(String companyName, String productSku, int quantity) {}

    public static class OrderProcessor {
        public static boolean process(CustomerOrder order) {
            return "Acme Corp".equals(order.companyName())
                && "WDG-99".equals(order.productSku())
                && order.quantity() == 5;
        }
    }

    void main() {
        // Harfi harfine şartnamedeki somut veri:
        var order = new CustomerOrder("Acme Corp", "WDG-99", 5);
        boolean success = OrderProcessor.process(order);

        assert success : "Somut veri doğrulaması başarısız!";
        System.out.println("✅ Somut veriyle çalışan deterministik test başarıyla geçti: " + order);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Test senaryolarında asla ‘geçerli bir kullanıcı adı’ veya ‘yeterli bakiye’ gibi soyut tabirler yazmayın; her veriyi ‘sarah.connor’ ve ‘500.00 TL’ şeklinde harfi harfine yazın.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never write abstract descriptors like ‘a valid user’ or ‘an active subscription’. Specify literal values (’sarah.connor’, ‘$500.00’) to guarantee deterministic tests.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Test senaryolarındaki veriler harfi harfine somut olmalıdır ('Acme Corp', '5 adet').
- Yer tutucular ('geçerli müşteri') AI'ın kafasına göre veri uydurmasına ve testlerin bozulmasına yol açar.
- Deterministik test verisi, şartnameden üretilen kodun her çalıştırmada aynı sonucu vermesini sağlar.

🎯 Key Architectural Takeaways

- Test data must be strictly literal ('Acme Corp', '5 items') with zero placeholders.
- Placeholders cause AI to improvise test values, introducing flaky and non-reproducible tests.
- Deterministic test fixtures guarantee identical outcomes across all code regenerations.

48

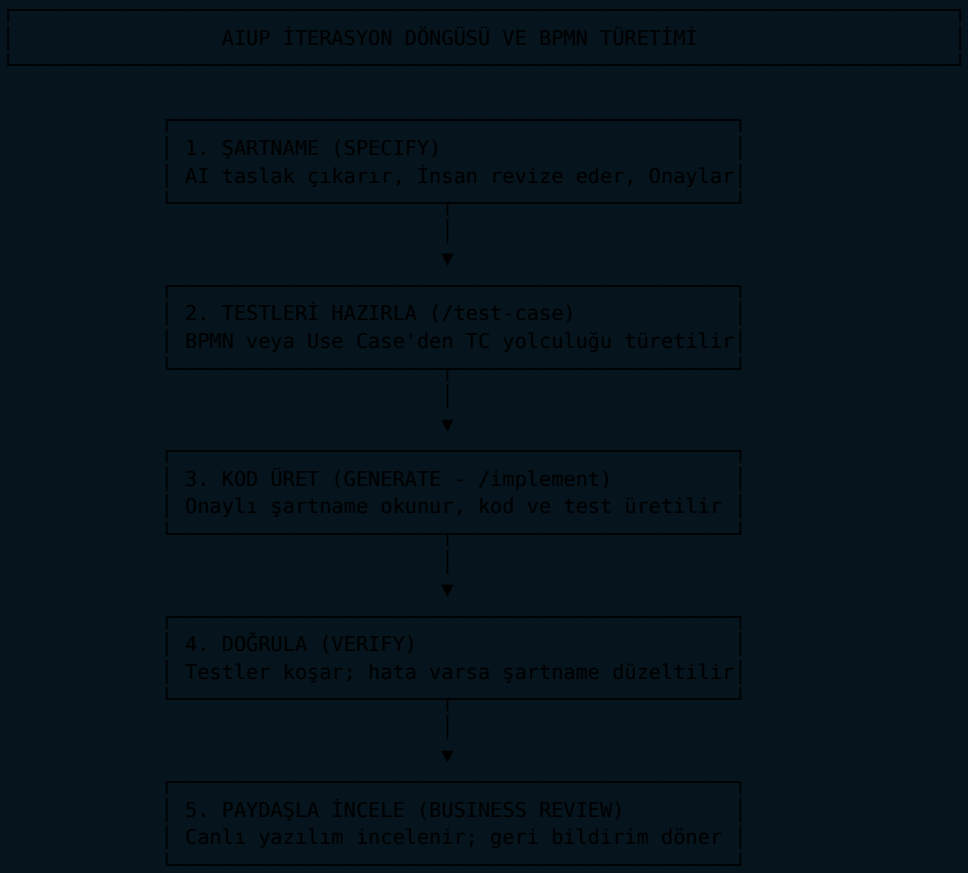
BÖLÜM: DILIMLER (SLICES), TESTLER VE İTERASYON DÖNGÜSÜ | MODULE: SLICES, TESTS, AND THE ITERATION LOOP

İterasyon Döngüsü ve BPMN Süreçlerinden Test Yolculuğu Türetme

The Iteration Loop & Deriving Journey Tests from BPMN Processes

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir film stüdyosunu düşünün: Önce senaryo yazılır (Şartname), sonra prova sahneleri planlanır (Testler), sonra aktörler sahneyi oynar (Kod üretimi), yönetmen sahneyi izler (Doğrulama), en son yapımcıya izletilir (Paydaş İncelemesi). Eğer yapımcı bir sahneyi beğenmezse aktör sette kafasına göre rol değiştirmez; senaryo yazarı senaryoyu günceller ve döngü baştan başlar!

🇬🇧 ENGLISH (ELI5)

Think of a film production studio: First the screenplay is polished (Spec), next the stunt sequences are choreographed (Tests), then actors perform on camera (Code Generation), the director reviews the footage (Verification), and finally the producer screens the cut (Business Review). If the producer wants a change, actors don't ad-lib on set; the writer updates the script and the loop re-executes cleanly!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

AIUP İterasyon Döngüsü ve BPMN Entegrasyonu: Her AI Unified Process iterasyonu 5 adımlı katı bir döngüde koşar: 1) Belirt (Specify): Analist şartnameyi hazırlar, paydaş onaylar (Approved). 2) Testleri Hazırla: `/test-case` komutu çalıştırılır. Eğer Inception aşamasında bir BPMN iş süreci çizilmişse, bu süreçteki her bir aktivite otomatik olarak bir use case çağrısına, her bir karar dalı ise ayrı bir TC yolculuk testine dönüştürülür! 3) Üret (Generate): `/implement UC-XXX` komutu mevcut kodları yerinde uzlaştırarak (reconcile) yeni kodları üretir. 4) Doğrula (Verify): Otomatik testler çalışır. Bir test başarısız olursa sorun %90 şartnamedeki bir belirsizlikten kaynaklanmaktadır; şartname düzeltilir ve tekrar üretilir. 5) Paydaşla İncele: Çalışan yazılım paydaşa sunulur; geri bildirimler doğrudan şartnameye işlenir. Bu döngüde 'şartnameyi atlayıp koda yama yapmak' (hotfix bypass) kesinlikle yasaktır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The 5-Step AIUP Iteration Loop & BPMN Derivation: Every engineering increment in the AI Unified Process executes a disciplined cadence: 1) Specify: The requirements engineer revises the use case to 'Approved'. 2) Prepare Tests: The `/test-case` engine ingests BPMN 2.0 process models from the Inception phase; every task activity maps to a use case invocation, and each gateway decision path synthesizes an independent TC journey file. 3) Generate: `/implement UC-XXX` executes, reconciling existing repositories and controllers in place without generating conflicting duplicates. 4) Verify: All tests execute automatically. Failures are traced back to spec ambiguities and rectified at the specification source. 5) Business Review: Working software is validated against the behavioral spec story. No code change ever bypasses this cycle.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 5 Adımlı İterasyon Döngüsünün Programatik Temsili
package com.maang.aiup.iteration;

public class IterationLoopDemo {
    public enum SpecStatus { DRAFT, APPROVED, IMPLEMENTED, VERIFIED, DONE }

    public record SpecLifecycle(String useCaseId, SpecStatus status, boolean testsPassing) {
        public SpecLifecycle advance() {
            return switch (status) {
                case DRAFT -> new SpecLifecycle(useCaseId, SpecStatus.APPROVED, false);
                case APPROVED -> new SpecLifecycle(useCaseId, SpecStatus.IMPLEMENTED, false);
                case IMPLEMENTED -> new SpecLifecycle(useCaseId, SpecStatus.VERIFIED, true);
                case VERIFIED -> testsPassing
                    ? new SpecLifecycle(useCaseId, SpecStatus.DONE, true)
                    : new SpecLifecycle(useCaseId, SpecStatus.APPROVED, false);
                case DONE -> this;
            };
        }
    }

    void main() {
        var cycle = new SpecLifecycle("UC-002", SpecStatus.DRAFT, false);
        while (cycle.status() != SpecStatus.DONE) {
            System.out.println("🔄 Döngü Adımı: " + cycle.status());
            cycle = cycle.advance();
        }
        System.out.println("🎯 İterasyon Tamamlandı: Use Case DONE durumuna ulaştı!");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Üretim baskısı altında bile 'kodu hemen düzeltelim, şartnameyi sonra güncelleriz' tuzağına düşmeyin! Şartnameyi atlayan her müdahale şartnamenin güvenilirliğini öldürür.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never fall into the trap of patching code directly under production pressure with promises to update the spec later. Bypassing the specification starts the rot.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- AIUP iterasyon döngüsü 5 adımdan oluşur: Specify -> Test -> Generate -> Verify -> Review.
- Inception aşamasındaki BPMN süreçleri otomatik olarak TC-XXX yolculuk testlerine dönüştürülür.
- Test başarısızlığı öncelikle şartnamedeki bir eksiklik olarak ele alınır ve şartnamede düzeltilir.

🎯 Key Architectural Takeaways

- The AIUP iteration loop runs 5 steps: Specify -> Test -> Generate -> Verify -> Review.
- BPMN business process models directly synthesize executable TC-XXX test journeys.
- Test failures indicate spec ambiguities first and are rectified at the specification source.